

Trade-offs & Production Readiness Analysis

Mal Credit Decision Data Platform | Part 2 | Priyanka

This document covers the key design trade-offs made in the Part 1 implementation — a medallion pipeline (bronze → silver → gold) that ingests credit bureau data, fraud scores, AML results, and internal customer profiles into a unified decision record. Each section references specific choices made in the codebase.

1. Conflict Resolution Trade-offs

How customer key conflicts are handled

The fundamental problem is that each source identifies the same customer differently — AECB uses an Emirates ID, the fraud provider uses phone and email, the AML provider uses name and date of birth, and our own database uses an internal UUID. I built a four-tier resolution cascade in `identity_resolver.py` that tries each key in order of reliability:

- Tier 1 — internal UUID (confidence 1.0): our own system, most trusted
- Tier 2 — Emirates ID (confidence 0.99): government-issued, exact match
- Tier 3 — phone + email together (confidence 0.90): both must match, one alone is not safe
- Tier 4 — name + date of birth fuzzy match (confidence 0.85): last resort, always flagged for review

Edge cases I thought through

- Same Emirates ID, different names — can happen after marriage or data entry errors. The conflict checker logs these to `silver.identity_resolution_exceptions` instead of silently overwriting.
- Shared phone numbers — family plans in UAE are common. This is why Tier 3 requires both phone AND email, not just one.
- Arabic name transliteration — 'Mohammed', 'Mohammad', 'Mohamed' are the same person but spelled differently across documents. I landed on a 0.85 similarity threshold after testing on around 200 sample name pairs. Below that, too many false matches appeared.
- Webhook arrives before profile — AML can screen a customer before their profile lands in bronze. Unmatched AML records sit in the exceptions table and are re-evaluated on the next pipeline run.

What I would change at 10x scale

- Move name matching out of PostgreSQL and into a dedicated entity resolution service (e.g. Splink or Zingg) — purpose-built for fuzzy matching at volume, much faster than SQL similarity.
- Add a Redis cache for UUID and Emirates ID lookups — most decisions are returning customers, so a cache hit avoids a database round-trip entirely.
- Replace the fixed 0.85 threshold with a trained probabilistic matching model once we have enough confirmed match/no-match pairs from production.

2. Data Quality Strategy

Why I split rules into MUST_PASS vs HIGH vs warnings

Not every data quality problem should stop the pipeline — that would block legitimate decisions during partial outages. The split I made is between rules that protect technical correctness and rules that protect business risk.

MUST_PASS rules (Emirates ID format, credit score range, fraud score in 0–1, AML risk level not null) are technical gates. If a fraud score comes back as -0.3, the scoring model breaks mathematically — there is no safe way to proceed. These halt the pipeline.

HIGH rules (AECB report within 90 days, fraud scored in last 24 hours, income above zero) are business risk signals. A 91-day-old AECB report is still usable data — it just means the decision gets a lower dq_score and routes to underwriter review instead of auto-approval. The pipeline continues.

Balancing decision speed vs data completeness

The key design choice here: decisions can proceed even when a non-critical source is degraded. Each gold decision snapshot carries a dq_score between 0 and 1 reflecting how complete the inputs were. A decision made with a stale AECB report gets a lower dq_score and lands in REFERRED state for manual review, rather than being blocked entirely.

What happens when AECB data is delayed

AECB delivers via SFTP once daily. If the file doesn't arrive by the pipeline window, affected applications move to REFERRED with a dq_flag of 'AECB_DATA_MISSING' and route to the manual underwriting queue. I deliberately chose this over auto-declining — declining because of a feed delay unfairly penalises customers and creates regulatory exposure. The pipeline distinguishes between 'we have bad data' (halt) and 'we have incomplete data' (continue with lower confidence and flag for human review).

3. Compliance & Auditability

How the design supports CBUAE audits

The core CBUAE requirement is that every credit decision must be fully reconstructable. Three choices in the Part 1 design make this possible:

- Append-only bronze tables — no UPDATE or DELETE is ever run on bronze. Every raw payload that arrived from any source is permanently stored with a timestamp and SHA-256 checksum.
- Immutable gold snapshots — `gold.credit_decision_inputs` stores a frozen copy of every input at decision time. Even if the customer's credit score changes tomorrow, the record shows exactly what it was when the decision was made.
- Full lineage chain — every gold row has foreign keys back to specific silver rows, and every silver row points to a specific bronze row. An auditor can trace any decision back to the original raw XML or JSON payload.

Files are also archived on S3 with KMS encryption and versioning enabled, retained for the 7-year period CBUAE requires for credit records.

Handling GDPR-style data deletion requests

This creates a genuine tension: a customer can request erasure of their personal data, but CBUAE requires credit records to be retained for 7 years. My approach resolves this through pseudonymisation rather than deletion:

- What gets erased: name, DOB, Emirates ID, phone, email — all PII fields — replaced with an irreversible hash in `silver.customer_master`.
- What gets retained: credit signals (scores, delinquency history, DTI ratios) and decision outcomes. These are financial records the bank is legally required to keep, not personal data in the regulatory sense.
- Bronze raw payloads: flagged with a deletion marker in S3 so they are excluded from future processing and reporting, while remaining available to regulators if subpoenaed.

4. Sharia Compliance Considerations

Where the current model stands

The Part 1 data model has a `product_type` field on `gold.credit_decision_inputs` covering `personal_finance`, `bnpl`, and `credit_card_alternative`. Extending this to distinguish Sharia-compliant variants is straightforward at the schema level — the more significant work is in the decision inputs themselves, because conventional credit signals don't translate directly to Islamic finance products.

Additional data points needed for Sharia products

- `profit_rate` instead of `interest_rate` — Murabaha charges a profit margin, not interest. Storing it as `interest_rate` misrepresents the product and fails a Shariah audit.
- `asset_type` and `asset_value_aed` — Ijara and Murabaha are asset-backed. The asset must exist and be Sharia-permissible (no alcohol, weapons, or prohibited business types).
- `financing_structure` enum — murabaha, ijara, musharaka, tawarruq each have different risk profiles and repayment patterns that the scoring model needs to handle separately.
- `shariah_board_approval_ref` — each product structure must be approved by a Shariah board. The reference ID ties the decision to the approved product design for audit purposes.
- `customer_banking_type` — Islamic banking vs conventional customer affects which products can be offered and which credit bureau signals are relevant.

Impact on existing credit signals

Some AECB signals are less predictive for Sharia customers. Credit card utilisation rate (`aecb_utilisation_rate`) is near-zero for customers who don't hold conventional credit cards — this field should not penalise them in the scoring model. Since each signal is stored separately per decision in the gold snapshot, the scoring layer can weight fields differently based on `financing_structure` without changing the data model.

5. What I Cut & Why — 48-Hour Scope Decisions

What was deliberately left out

- Kafka / real-time streaming — AECB is batch and fraud scoring is synchronous per request. Streaming adds infrastructure complexity with no throughput benefit at 10K/day. Revisit at 100K+/day if source latency becomes a bottleneck.
- Debezium CDC — replaced with a simpler updated_at watermark approach for customer profile extraction. Debezium requires a managed Kafka cluster and careful offset management — too much operational overhead for launch when a watermark gets us 95% of the way there.
- Column-level PII encryption (Emirates ID, DOB) — the schema is designed for it (dedicated columns, not mixed into JSONB), but encryption adds key management complexity. Getting this wrong at launch is a bigger risk than a short delay. Planned for Day 60.
- ML feature store — a rule-based credit policy runs fine on raw silver fields at launch. Building a feature store before we have 30+ days of production data would be premature and likely wrong.
- Multi-region disaster recovery — single region with Multi-AZ RDS covers realistic failure modes at launch. Multi-region adds cost and complexity that isn't justified until we scale beyond UAE.

First five post-launch improvements, in priority order

- GIN trigram index on silver.customer_master.full_name — name matching currently does a sequential scan. A GIN index cuts this to near-constant time. Highest value, lowest effort.
- Column-level encryption for Emirates ID and DOB — before the Day 60 compliance checkpoint.
- Velocity fraud alerting — flag any customer_key appearing in more than 3 applications within 7 days. The data is already there in gold; it just needs a view and a DQ rule.
- Sharia product type support — add the six Sharia-specific fields to the gold snapshot and extend product_type to cover murabaha, ijara, musharaka.
- Automated CBUAE monthly report — all the data exists in gold already; needs a scheduled Athena query and a PDF export to replace what would otherwise be a manual process.